

ChemML - Sistema de Predição de Propriedades Moleculares

Visão Geral

ChemML é um sistema completo de predição de propriedades moleculares que combina machine learning com química computacional. O sistema permite que usuários insiram moléculas (via SMILES) e obtenham predições de propriedades físico-químicas importantes.

Objetivos do Projeto

- **Técnico:** Demonstrar aplicação de ML em problemas científicos reais
- **Profissional:** Mostrar habilidades em Flask, PyTorch, APIs REST e bancos de dados
- **Diferencial:** Aplicar conhecimento químico para feature engineering e validação

Arquitetura do Sistema

```
chemml/
├── app/
│   ├── __init__.py
│   ├── models/
│   │   ├── __init__.py
│   │   ├── ml_models.py    # Modelos PyTorch
│   │   ├── database.py     # Modelos SQLAlchemy
│   │   └── molecular.py    # Processamento molecular
│   ├── routes/
│   │   ├── __init__.py
│   │   ├── api.py          # Endpoints da API
│   │   ├── auth.py         # Autenticação
│   │   └── web.py          # Interface web
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── descriptors.py  # Cálculo de descritores
│   │   ├── validators.py   # Validação de entrada
│   │   └── preprocessing.py # Pré-processamento
│   └── templates/
│       ├── base.html
│       ├── index.html
│       ├── predict.html
│       └── history.html
├── data/
│   ├── raw/                # Dados brutos
│   ├── processed/          # Dados processados
│   └── models/              # Modelos treinados
├── notebooks/
│   ├── 01_data_exploration.ipynb
│   ├── 02_model_development.ipynb
│   └── 03_model_evaluation.ipynb
├── tests/
├── config.py
├── requirements.txt
└── README.md
```

Funcionalidades Principais

1. Predição de Propriedades

- **Solubilidade em água:** Modelo de regressão para LogS
- **Toxicidade:** Classificação binária para toxicidade aguda
- **Ponto de fusão:** Regressão para temperatura de fusão
- **Biodegradabilidade:** Classificação para degradação ambiental

2. Interface Web

- Upload de moléculas via SMILES ou desenho molecular
- Visualização 2D/3D das moléculas
- Histórico de predições por usuário
- Dashboard com estatísticas

3. API REST

- Endpoints para integração com outros sistemas
- Autenticação JWT
- Documentação automática com Swagger
- Rate limiting e caching

4. Sistema de Validação

- Validação química de SMILES
- Avaliação de confiabilidade das predições
- Detecção de moléculas fora do domínio de aplicação



Datasets Sugeridos

1. Solubilidade:

- AqSolDB (10,000+ compostos)
- ESOL dataset (1,100+ compostos)

2. Toxicidade:

- Tox21 Challenge data
- ChEMBL bioactivity data

3. Ponto de Fusão:

- Bradley Melting Point dataset
- NIST Chemistry WebBook



Stack Tecnológica

Backend

- **Flask**: Framework web
- **SQLAlchemy**: ORM para banco de dados
- **PyTorch**: Machine learning
- **RDKit**: Processamento molecular

- **Pandas/NumPy**: Manipulação de dados

Frontend

- **HTML/CSS/JavaScript**: Interface básica
- **Chart.js**: Visualizações
- **Bootstrap**: Design responsivo

Banco de Dados

- **SQLite**: Desenvolvimento
- **PostgreSQL**: Produção (opcional)



Cronograma de Desenvolvimento

Fase 1: Fundação (Semana 1-2)

- ☐ Configuração do ambiente e estrutura do projeto
- ☐ Coleta e preparação dos datasets
- ☐ Implementação das funcionalidades básicas de processamento molecular
- ☐ Criação do esquema do banco de dados

Fase 2: Modelos ML (Semana 3-4)

- ☐ Desenvolvimento dos modelos de predição
- ☐ Treinamento e otimização de hiperparâmetros
- ☐ Implementação de métricas de avaliação
- ☐ Criação de pipeline de validação

Fase 3: API e Backend (Semana 5-6)

- ☐ Implementação da API REST
- ☐ Sistema de autenticação
- ☐ Integração dos modelos com a API
- ☐ Testes unitários e de integração

Fase 4: Frontend (Semana 7-8)

- ☐ Interface web para predições
- ☐ Dashboard de usuário
- ☐ Visualizações moleculares
- ☐ Sistema de histórico

Fase 5: Finalização (Semana 9-10)

- ☐ Documentação completa

- Deploy e configuração
- Otimizações de performance
- Testes finais

Modelos de Machine Learning

1. Modelo de Solubilidade

python

```
class SolubilityPredictor(nn.Module):
    def __init__(self, input_size=2048):
        super().__init__()
        self.fc1 = nn.Linear(input_size, 512)
        self.fc2 = nn.Linear(512, 256)
        self.fc3 = nn.Linear(256, 128)
        self.fc4 = nn.Linear(128, 1)
        self.dropout = nn.Dropout(0.3)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.dropout(x)
        x = torch.relu(self.fc2(x))
        x = self.dropout(x)
        x = torch.relu(self.fc3(x))
        x = self.fc4(x)
        return x
```

2. Features Moleculares

- **Fingerprints:** Morgan, MACCS, AtomPair
- **Descritores:** MW, LogP, TPSA, HBD, HBA
- **Estruturais:** Número de anéis, heteroátomos, centros quirais

Métricas de Avaliação

Regressão (Solubilidade, Ponto de Fusão)

- **R²:** Coeficiente de determinação
- **RMSE:** Erro médio quadrático
- **MAE:** Erro médio absoluto

Classificação (Toxicidade, Biodegradabilidade)

- **Accuracy:** Precisão geral
- **F1-Score:** Harmônica entre precisão e recall

- **AUC-ROC:** Área sob a curva ROC

Deployment

Desenvolvimento

```
bash
```

```
# Configuração do ambiente
```

```
python -m venv venv
```

```
source venv/bin/activate # Linux/Mac
```

```
pip install -r requirements.txt
```

```
# Inicialização do banco
```

```
flask db init
```

```
flask db migrate
```

```
flask db upgrade
```

```
# Execução
```

```
python app.py
```

Produção

- **Docker:** Containerização
- **Gunicorn:** Servidor WSGI
- **Nginx:** Proxy reverso
- **Redis:** Cache e sessões

Estrutura do Banco de Dados

sql

-- Tabela de usuários

```
CREATE TABLE users (  
  id INTEGER PRIMARY KEY,  
  username VARCHAR(80) UNIQUE NOT NULL,  
  email VARCHAR(120) UNIQUE NOT NULL,  
  password_hash VARCHAR(128) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Tabela de moléculas

```
CREATE TABLE molecules (  
  id INTEGER PRIMARY KEY,  
  smiles VARCHAR(500) NOT NULL,  
  molecular_weight REAL,  
  logp REAL,  
  tpsa REAL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Tabela de predições

```
CREATE TABLE predictions (  
  id INTEGER PRIMARY KEY,  
  user_id INTEGER REFERENCES users(id),  
  molecule_id INTEGER REFERENCES molecules(id),  
  property_type VARCHAR(50) NOT NULL,  
  predicted_value REAL,  
  confidence REAL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Pontos de Destaque para Recrutadores

1. **Aplicação Real:** Resolve problemas genuínos da indústria farmacêutica
2. **Stack Completa:** Backend, ML, API, frontend e banco de dados
3. **Conhecimento Científico:** Demonstra expertise em química computacional
4. **Escalabilidade:** Arquitetura preparada para crescimento
5. **Qualidade:** Testes, documentação e boas práticas

Próximos Passos

1. **Melhorias nos Modelos:** Implementar redes neurais mais sofisticadas
2. **Mais Propriedades:** Adicionar predições de outras propriedades

3. **ML Explicável:** Implementar técnicas de interpretabilidade
4. **Performance:** Otimizar para previsões em tempo real
5. **Integração:** APIs para integrar com outros sistemas

Links Úteis

- **RDKit Documentation:** <https://www.rdkit.org/docs/>
- **PyTorch Tutorials:** <https://pytorch.org/tutorials/>
- **Flask Documentation:** <https://flask.palletsprojects.com/>
- **ChEMBL Database:** <https://www.ebi.ac.uk/chembl/>

Este projeto demonstra a aplicação prática de machine learning em química, combinando conhecimento científico com habilidades técnicas avançadas.